

Journée 4

Architecture Micro-services

S14 · TP CI/CD : pipeline automatisé du service Avis-Clients

Nom / Prénom : DANILO Elouan

Service : **Avis-Clients**

Dépôt : `https://github.com/Oasio/avis-clients-cicd`

Image : `ghcr.io/oasio/avis-clients-cicd:latest`

1. Contexte

Le TP consiste à doter mon service **Avis-Clients** (Symfony 7 + API Platform) d'un **pipeline CI/CD complet** : à chaque `push`, on lance les tests, on construit une image Docker, et on la publie dans un *Container Registry*, prête à déployer.

Le polycopié s'appuie sur **GitLab.com**. J'ai réalisé le TP sur **GitHub**, l'équivalent présenté en cours (S13.1) : **GitHub Actions** pour le pipeline (fichier `.github/workflows/ci.yml`) et le **GitHub Container Registry** (`ghcr.io`) pour l'image. La logique reste identique : stages séquentiels, tag par hash de commit, déploiement manuel. Les principales adaptations : le runner GitHub fournit déjà Docker (pas besoin de *docker-in-docker*), et le `when: manual` de GitLab est reproduit par un **environnement protégé**.

2. Le pipeline mis en place

Le workflow se déclenche **automatiquement à chaque push sur main** (et manuellement via `workflow_dispatch`). Il enchaîne quatre stages :

- **test** : PHPUnit sur une base SQLite jetable ;
- **build** : construction de l'image Docker à partir du `Dockerfile` ;
- **package** : push de l'image sur `ghcr.io` (tags : `<sha>` et `:latest`) ;
- **deploy** (*bonus*) : déploiement simulé, déclenché manuellement.

Stage TEST

Les tests d'API persistent réellement des avis : il faut donc une base. Plutôt que de lancer un serveur MariaDB dans la CI, je surcharge `DATABASE_URL` vers une base **SQLite** jetable, et je crée le schéma depuis les métadonnées de l'entité.

```
test:
  runs-on: ubuntu-latest
  container: { image: php:8.3-cli }
  env:
    APP_ENV: test
    DATABASE_URL: "sqlite:///kernel.project_dir%/var/data_test.db"
  steps:
    - uses: actions/checkout@v4
    - run: docker-php-ext-install intl pdo_mysql pdo_sqlite zip
    - run: composer install --no-interaction --prefer-dist
    - run: php bin/console doctrine:schema:create --no-interaction
    - run: php bin/phpunit # 8 tests au vert
```

Stages BUILD et PACKAGE

Le stage `build` vérifie que l'image se construit. Le stage `package` se connecte au registry avec le `GITHUB_TOKEN` auto-généré (aucun identifiant en dur) puis pousse l'image taguée par le hash court du

commit *et* par latest.

```
package:
  needs: build
  permissions: { packages: write }
  steps:
    - run: echo "$GITHUB_TOKEN" | docker login ghcr.io -u $ACTOR --password-stdin
    - run: |
        IMAGE=$(echo "$IMAGE_NAME" | tr '[:upper:]' '[:lower:]') # ghcr = minuscules
        docker build -t "$IMAGE:${GITHUB_SHA:0:7}" -t "$IMAGE:latest" .
        docker push "$IMAGE:${GITHUB_SHA:0:7}"
        docker push "$IMAGE:latest"
```

Stage DEPLOY (bonus)

GitHub Actions n'a pas de `when: manual` par job. Je reproduis le comportement avec un **environnement production** protégé par un *required reviewer* : le job `deploy` reste en attente tant qu'il n'est pas approuvé dans l'interface. Le déploiement lui-même est simulé (un `echo`), aucun serveur n'étant requis pour le TP.

3. Problèmes rencontrés et solutions

Problème	Solution mise en place
Les tests d'API (WebTestCase) écrivent en base ; aucun serveur MariaDB n'est disponible dans le runner CI.	Surcharge de DATABASE_URL vers une base SQLite jetable dans le job. Les variables d'environnement réelles étant prioritaires sur les fichiers <code>.env</code> , la config MariaDB du projet reste intacte.
La migration livrée (Version20260612000000) est du SQL MariaDB pur (AUTO_INCREMENT, ENGINE=InnoDB), non rejouable sur SQLite.	En CI, création du schéma via <code>doctrine:schema:create</code> (généré depuis les métadonnées de l'entité, indépendant du SGBD) au lieu de rejouer la migration.
Le registry <code>ghcr.io</code> refuse les noms d'image en majuscules, or <code>github.repository</code> contient <code>Oasio</code> .	Passage en minuscules dans le script avec <code>tr '[:upper:]' '[:lower:]'</code> avant chaque <code>docker build/push</code> .
Authentification au registry sans écrire de secret.	Login via le <code>GITHUB_TOKEN</code> auto-généré et permission <code>packages: write</code> sur le job, soit l'équivalent des variables <code>\$CI_REGISTRY_*</code> de GitLab.
Reproduire le <code>when: manual</code> de GitLab, absent de GitHub Actions.	Job <code>deploy</code> rattaché à un environnement production protégé par un <i>required reviewer</i> : il ne démarre qu'après approbation manuelle.

4. Résultat

Le dernier pipeline passe **intégralement au vert** (statut *Success*, 5 min 15 s), et l'image est bien publiée dans le registry, taguée `latest` et par le hash du commit. Le déclenchement est automatique à chaque `push` sur `main`, et tous les stages attendus (`test`, `build`, `package`) sont exécutés, plus le déploiement manuel en bonus.

5. Conclusion

Mon service Avis-Clients dispose désormais d'un pipeline CI/CD versionné qui, à chaque `push`, teste le code, construit l'image Docker et la publie sur `ghcr.io`, prête à déployer. Les deux points qui m'ont demandé le plus d'ajustement, faire tourner les tests sans serveur de base et publier proprement sur le registry, ont été réglés respectivement par une base SQLite jetable (schéma créé depuis l'entité) et par un login au registry via le jeton auto-généré. Le déploiement manuel via environnement protégé apporte le point bonus tout en évitant tout déploiement accidentel.

A. Annexes : captures d’écran

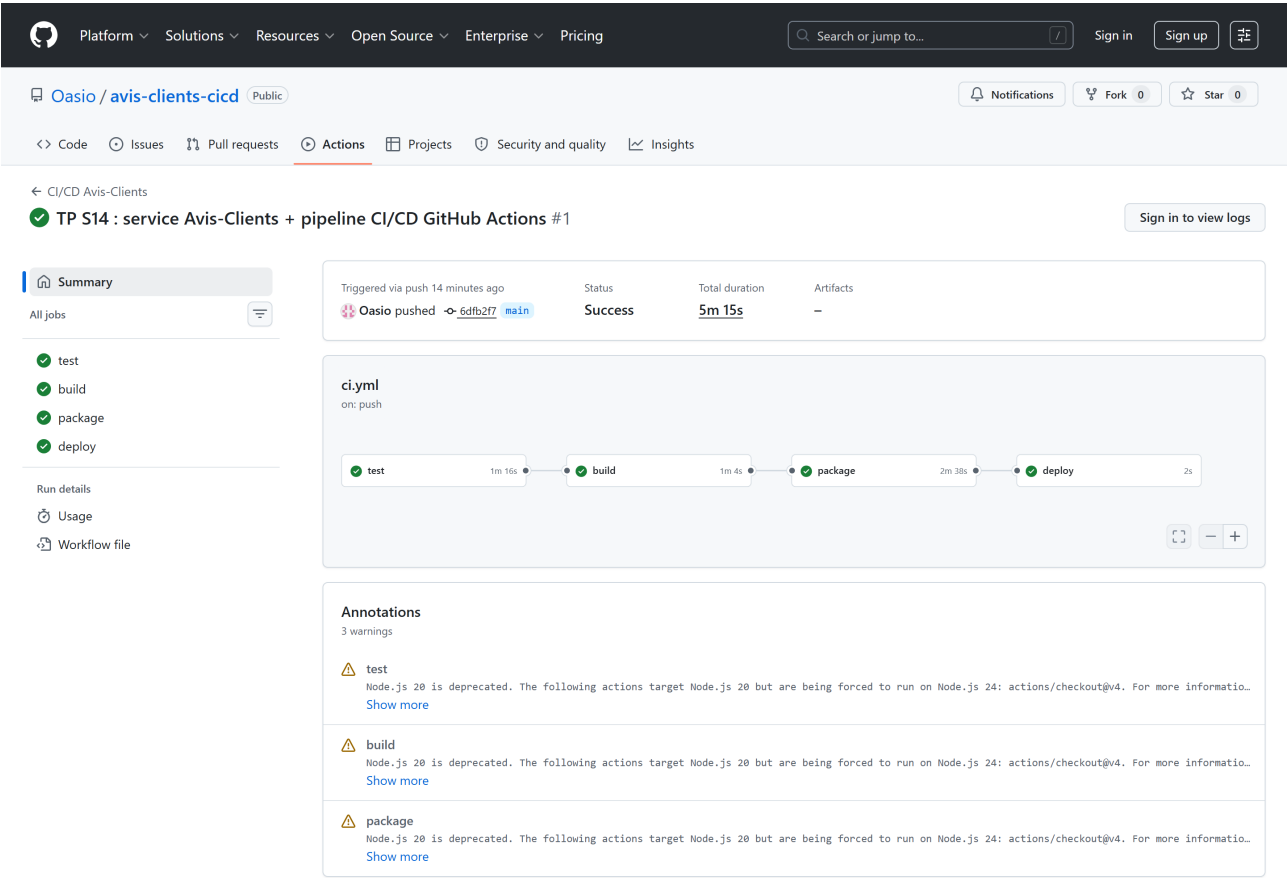


FIGURE 1 – Annexe A. Dernier pipeline GitHub Actions en **succès** (Success, 5 min 15 s). Les quatre stages test → build → package → deploy sont verts ; déclenché via push sur main.

PlatformSolutionsResourcesOpen SourceEnterprisePricing

Search or jump to...Sign inSign up

Oasio / avis-clients-cicdPublic

NotificationsFork0Star0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

avis-clients-cicdlatestPublicLatest

Install from the command lineLearn more about packages

\$ docker pull ghcr.io/oasio/avis-clients-cicd:latest

Recent tagged image versions

latest6dfb2f7

Published 13 minutes ago · Digest

0

View all tagged versions

README.md

Avis-Clients — Service Symfony (FlexShop)

2^e service commun du projet micro-services FlexShop. Gère les avis (Review) sur les produits du service Catalogue (Dianao). Construit avec Svmfony 7 + API Platform.

Details

Oasio

avis-clients-cicd

0 stars

Last published13 minutes ago0

Issues

Total downloads0

Contributors1

Oasio Elouan

Open an issue

FIGURE 2 – Annexe B. GitHub Container Registry (ghcr.io) : l’image avis-clients-cicd est publiée et taguée latest (et par le hash court du commit). Commande de récupération : docker pull ghcr.io/oasio/avis-clients-cicd:latest.